

BUSN 41210: Financial Analytics

Lecture 8: Trees, Random Forests, and Boosted Trees

Dacheng Xiu

Chicago Booth

September 23, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Trees

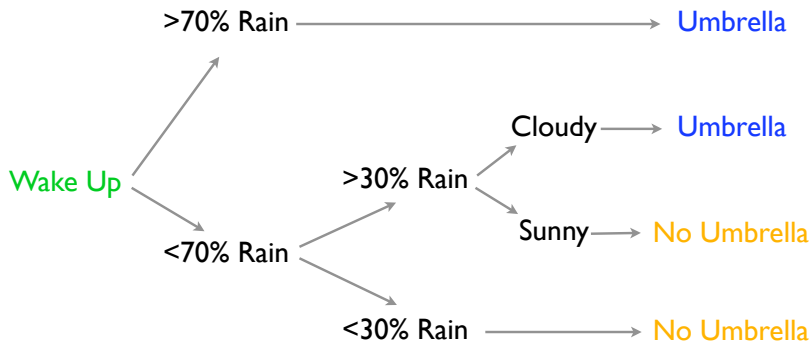
Part I — one tree

- ▶ **Classification And Regression Trees**: using tree-logic to make predictions
- ▶ Growing a tree: deviance, impurity, and the greedy split
- ▶ Pruning: cost complexity, and choosing α out of sample
- ▶ Nonlinearity and interaction, for free

Part II — many trees

- ▶ Bagging, and the one extra trick that makes it a **random forest**
- ▶ **Boosting**: fit the residual, shrink, repeat
- ▶ Which model actually wins, and opening the black box

What is a Decision Tree?



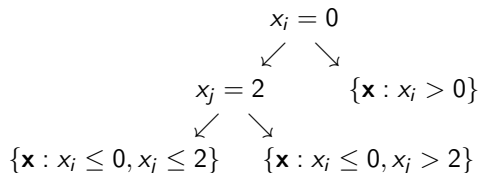
Tree-logic uses a series of steps to come to a conclusion.

The trick is to have mini-decisions combine for good choices.

Each decision is a node, and the final prediction is a **leaf node**

Decision Trees are Like a Game of Mousetrap

You drop your $\mathbf{x}$ covariates in at the top, and each decision node bounces you either left or right. Finally, you end up in a **leaf node** which contains the data subset defined by these decisions (splits).



Tree-based Statistical Learning

Tree-based learning: predicting outcome y from predictors $x = (x_1, \dots, x_p)'$ by dividing up the feature space into small regions where the outcomes are more similar.

Within each region, a very simple model is fit locally.

This works both when y is categorical and continuous, i.e., both for **classification** and **regression**

Regions can be achieved by making successive binary splits on the predictors variables $x_1, \dots, x_p$, i.e. we choose a variable $x_j, j = 1, \dots, p$, **divide** up the feature space according to

$$x_j \leq c \quad \text{and} \quad x_j > c$$

Then we repeat the same on each half.

Estimation of Decision Trees

As usual, we'll maximize data likelihood (minimize deviance).

But what are the observation probabilities in a tree model?

Two types of likelihood: **classification** and **regression** trees.

A given covariate $\mathbf{x}$ dictates your path through tree nodes, leading to a **leaf node** at the end.

Classification trees have **class probabilities** at the leaves.

Probability I'll be in heavy rain is 0.9 (so take an umbrella).

Regression trees have a **mean response** at the leaves.

The expected amount of rain is 2in (so take an umbrella).

Tree Deviances

Regression Deviance: $\sum_{i=1}^n (y_i - \hat{y}_i)^2$

Classification Deviance: $-\sum_{i=1}^n \log(\hat{p}_{y_i})$, where $\hat{p}_{y_i} = \hat{p}(y_i|x_i)$.

It is also common to use Gini Deviance, $-\sum_{i=1}^n \hat{p}_{y_i}(1 - \hat{p}_{y_i})$, a measure of multinomial variance.

Instead of being based on $\mathbf{x}'\beta$, predicted $\hat{p}$ and $\hat{y}$ are functions of $\mathbf{x}$ passed through the decision nodes.

We need a way to estimate the sequence of decisions.

- How many are they? What is the order?

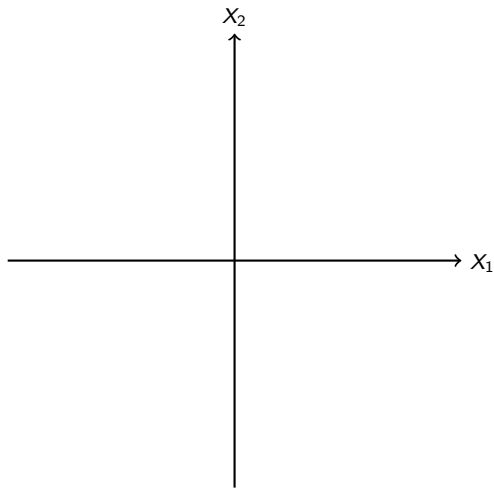
There is a *huge* set of possible tree configurations.

Classification Trees

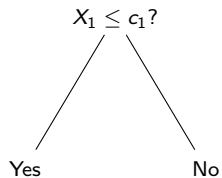
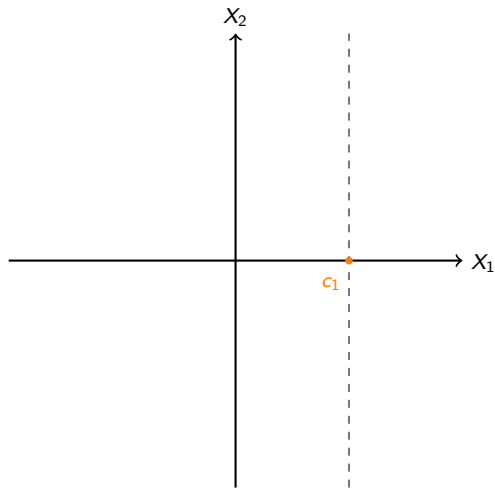
Classification trees are popular because they are interpretable (they mimic how decisions are made).

- ▶ In classification, $y_i \in 1, \dots, K$ are the class labels, and $x_i \in R_p$ measure the p predictor variables.
- ▶ The classification tree can be thought of as defining m regions (rectangles) $R_1, \dots, R_m$, each corresponding to a leaf of the tree
- ▶ We assign each R_j a class label $c_j \in 1, \dots, K$ (typically the most dominant class within the region).
- ▶ We then classify a new point x as c_j if it falls in region R_j .

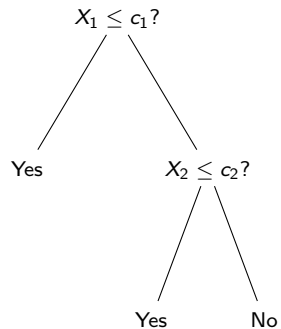
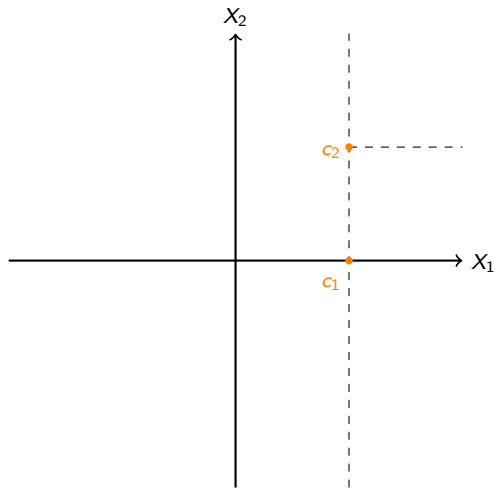
Classification Trees: An Illustration



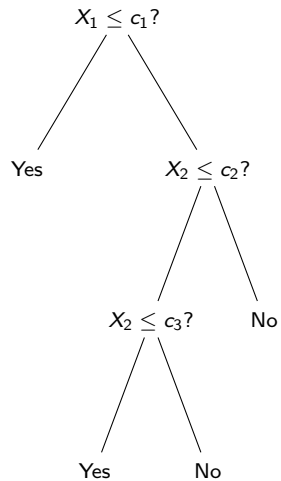
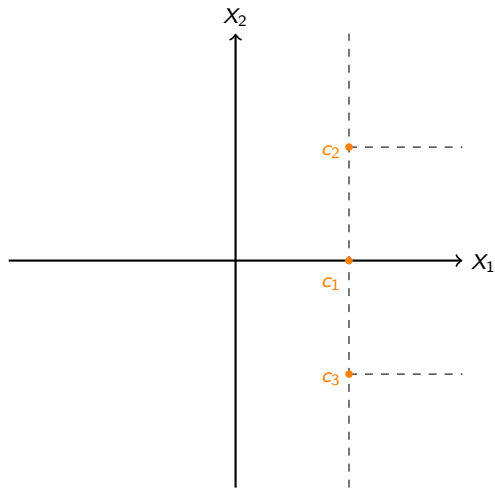
Classification Trees: An Illustration



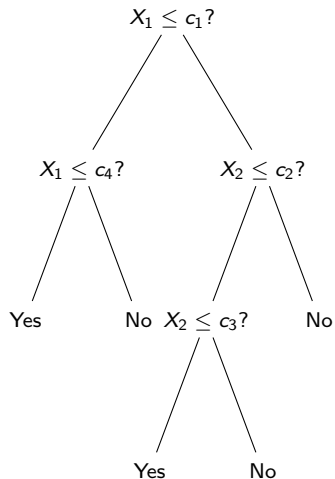
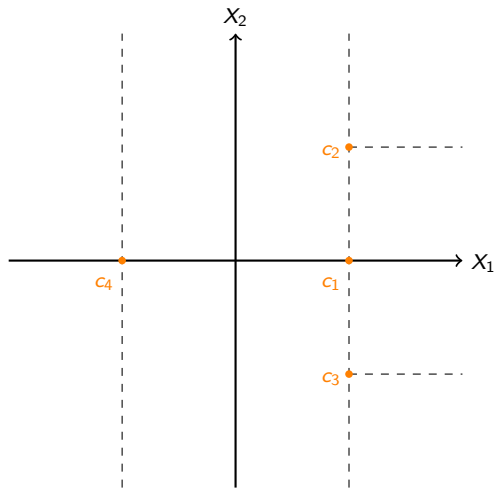
Classification Trees: An Illustration



Classification Trees: An Illustration

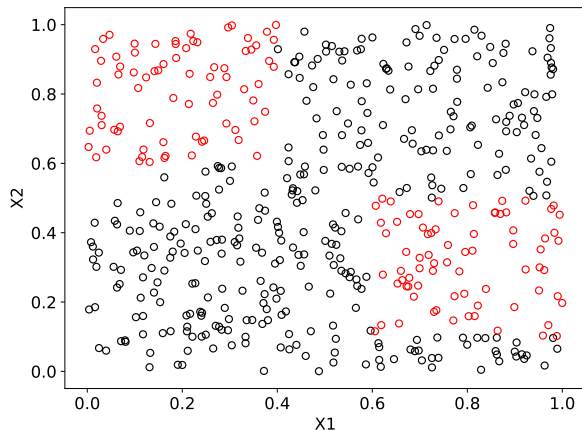


Classification Trees: An Illustration



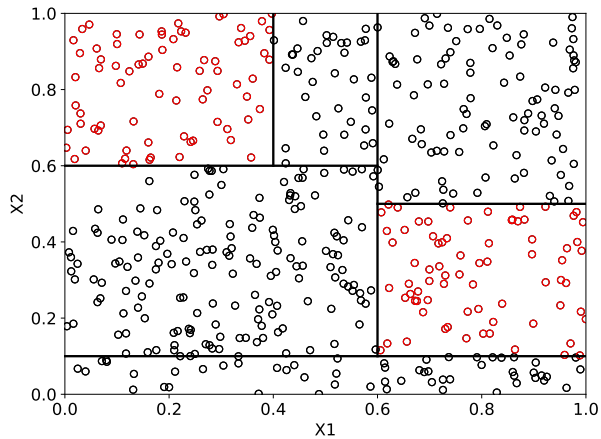
Toy Example: Classification Tree

Example: $n = 500$ points in $p = 2$ dimensions, falling into classes 0 and 1, as marked by colors

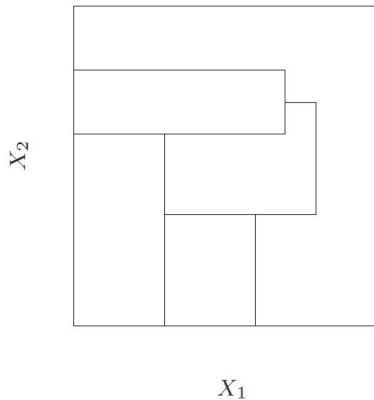


Does dividing up the feature space into rectangles look like it would work here?

Toy Example: Classification Tree



Example: Other Regions Not Allowed



Predicted Class Probabilities

With classification trees, we get not only the predicted classes for new points but also the **predicted class probabilities**.

Note that each region R_j contains some subset of the training data (x_i, y_i) , say, n_j points.

Further, for each class $k = 1, \dots, K$, we can estimate the probability that the class label is k given that the feature vector lies in region R_j

$$\hat{P}(C = k | X \in R_j) = \frac{\#y_i : y_i \in R_j \text{ and } y_i = k}{\#y_i : y_i \in R_j}$$

the **proportion of points** in the region that are of class k .

The predicted class $\hat{c}_j$ is **the most common occurring class** among these points

$$\hat{c}_j = \arg \max_{k=1, \dots, K} \hat{P}(C = k | X \in R_j)$$

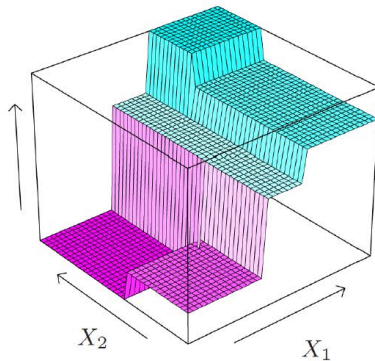
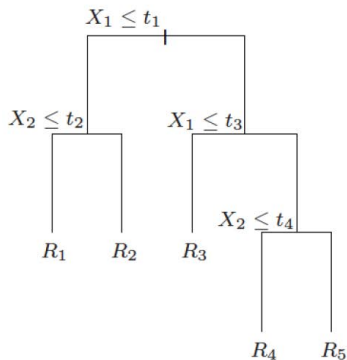
Classification Trees and Their Competitors

	Model assumptions?	Estimated probabilities?	Interpretable?	Flexible?
LR	Yes	Yes	Yes	No
k-NN	No	No	No	Yes
Trees	No	Yes	Yes	Somewhat

	Predicts well?
LR	Depends on X
k-NN	If properly tuned
Trees	?

Regression Trees

Suppose that now we want to predict a continuous outcome instead of a class label. Essentially, everything follows as before, but now we just fit a mean of a continuous outcome rather than a proportion inside each rectangle



Regression Trees

The estimated regression function has the form

$$E[y|x] = \sum_{j=1}^m c_j \cdot \mathbb{1}\{x \in R_j\} = c_j \text{ such that } x \in R_j,$$

just as it did with classification. The quantities c_j are no longer predicted classes, but instead they are real numbers.

How would we choose c_j ? Just take the average response of all of the points in the region,

$$c_j = \frac{1}{n_j} \sum_{x_i \in R_j} y_i$$

The main difference in building the tree is that we use **sums of squares** instead of misclassification error (or Gini index or deviance) to decide which region to split.

How to Build Trees?

There are two main issues to consider in building a tree:

- ▶ How to choose the splits?
- ▶ How big to grow the tree?

Think first about varying the depth of the tree ... which is more complex, a big tree or a small tree? What tradeoff is at play here? How might we eventually consider choosing the depth?

Now for a fixed depth, consider choosing the splits. If the tree has depth d , then it has about 2^d nodes. At each node we could choose any of p the variables for the split—this means that the number of possibilities is

$$p \cdot 2^d$$

This is **huge** even for moderate d ! And we haven't even counted the actual split points themselves

CART

We estimate decision trees by being recursive and greedy

CART grows the tree through a sequence of splits:

- ▶ Given any set (node) of data, you can find the **optimal split** (the error minimizing split) and divide into two child sets.
- ▶ We then look at each child set, and again find the optimal split to divide it into two homogeneous subsets.
- ▶ The children become parents, and we look again for the optimal split on their new children (the grandchildren!).

You stop splitting and growing when the size of the leaf nodes hits some minimum threshold (e.g., say no less than 10 obsv per leaf).

Often there are also minimum deviance improvement thresholds.

NBC Example

Data from NBC on response to TV pilots. 6241 views and 20 questions for 40 shows. Primary goal is predicting **engagement**.

Classic measures of broadcast marketability are **Ratings**.

- ▶ **GRP**: gross ratings points; estimated total viewership.

Projected Engagement: a more subtle measure of audience. After watching a show, viewer is quizzed on order and detail. This measures their engagement with the show (and ads!).

Viewer demographics: These demographics include percent of viewership for each show that falls in a number of categories defined by region, race, and how the household consumes TV.

Use the DecisionTreeClassifier for CART in python

```
from sklearn import tree
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, Y)
```

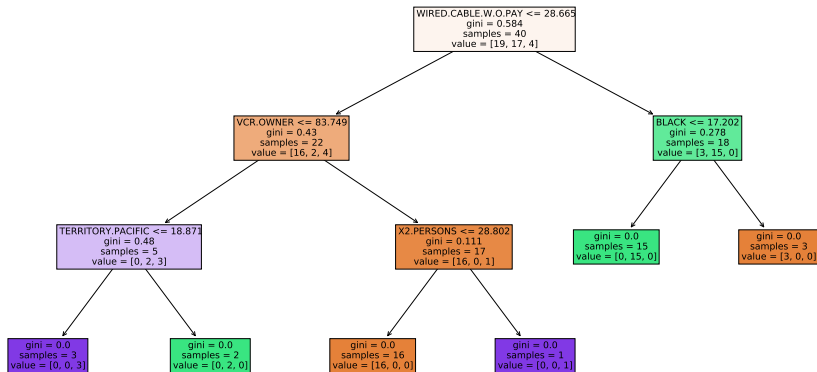
As usual, you can `plot`, and `predict` the tree.

Consider a classification tree to predict `genre` from `demographics`.

```
X = demos.iloc[:,1:]
y = nbc.Genre
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, Y)
```

Trees are easiest to understand in a **dendrogram**

Shows the sequence of internal splits, ending in leaf-node decisions.



To get the dendrogram, do `tree.expors_text(clf)` then `plot_tree(clf)`.

Consider predicting **engagement** from **ratings** and **genre**.

Split on genre by turning it into a group of numeric variables:

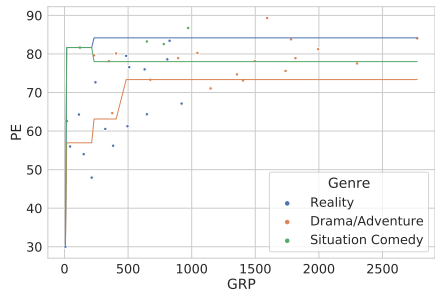
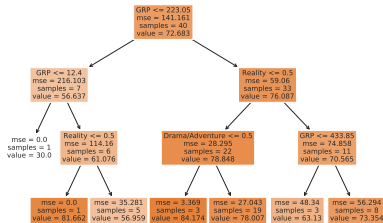
```
X = pd.concat([nbc.GRP, pd.get_dummies(nbc.Genre)], axis=1)
```

A regression tree:

```
X = pd.concat([nbc.GRP, pd.get_dummies(nbc.Genre)], axis=1)
y = nbc.PE
clf = tree.DecisionTreeRegressor().fit(X, y)
```

Instead of genre, leaf predictions are expected engagement.

An NBC Show Engagement Tree



Nonlinear: PE increases with GRP, but in jumps

Follow how the tree translates into changing $E[PE]$

Trees provide **Automatic Interaction Detection**

For example, different genres are more/less dependent on GRP.

AID was an original motivation for building decision trees. Older algorithms have it in their name: CHAID, ...

This is pretty powerful technology: **nonlinearity** and **interaction** without having to specify it in advance. Moreover, nonconstant variance is no problem.

Methods with these characteristics are called **nonparametric**.

No assumed parametric model (eg, $y = x\beta + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$).

Pruning via Minimal Cost-complexity Algorithm

Biggest challenge with such flexible models is avoiding overfit. For CART, the usual solution is to rely on out-of-sample prediction.

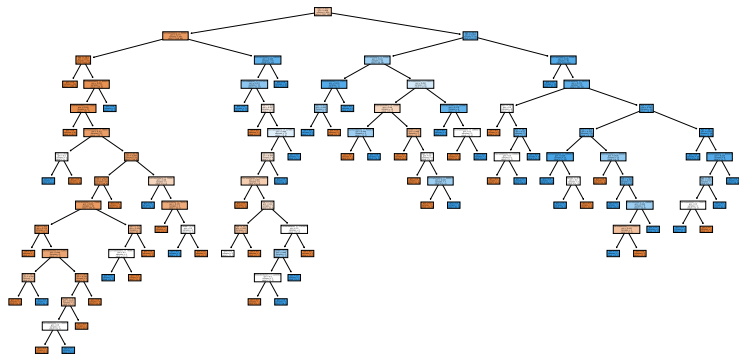
The minimal cost-complexity pruning algorithm, like LASSO, is parametrized by a complexity parameter α .

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

Pruning yields candidate trees, and we use a testing sample to choose.

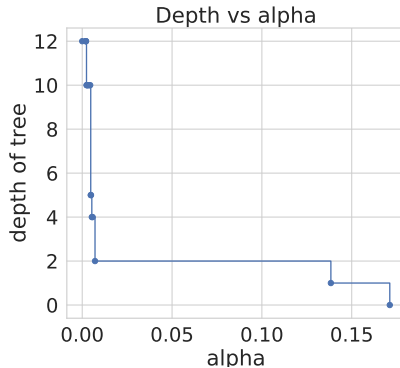
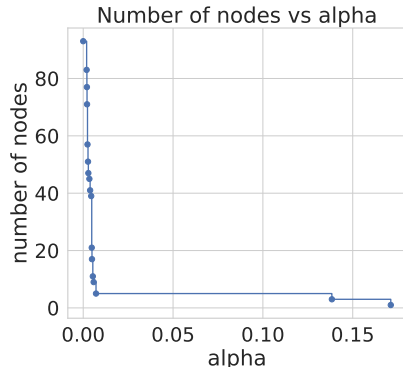
Example: Social Network Ads

A categorical dataset to determine whether a user purchased a particular product. Gender, age and estimated salary are available to help in predicting purchase behavior. An overly complex tree fit to 400 clients:

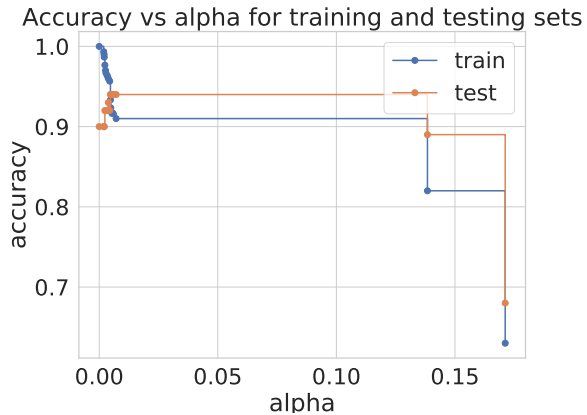


Do we need all the splits? Is the tree just fitting noise?

Complexity as a Function of Tuning Parameter

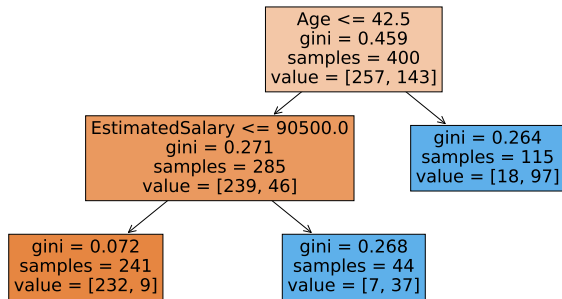


Out-of-sample Tree Pruning Accuracy



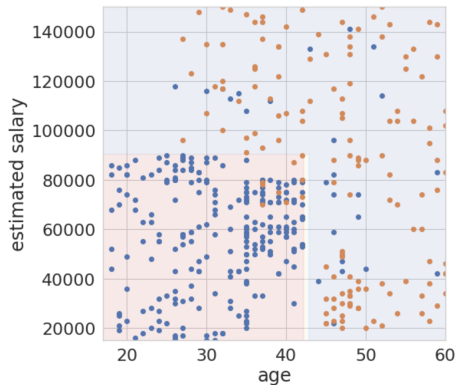
We thus fit a tree with depth 2:

```
clf = tree.DecisionTreeClassifier(max_leaf_nodes=3).fit(X, Y)
```



CV chooses age and estimated salary as deciding variables.

Social Network Ads Tree



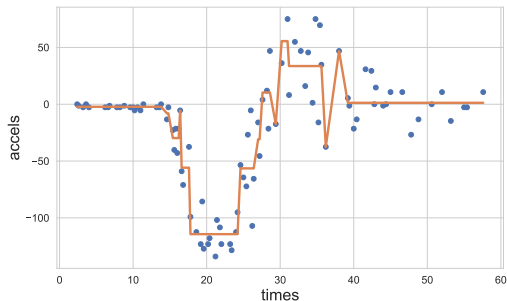
With only 2 relevant inputs, we can plot the data and tree fit.

Trees Detect Nonlinearity

They automatically learn non-linear response functions and will discover interactions between variables.

Example: Motorcycle Crash Test Dummy Data

x is time from impact, y is acceleration on the helmet.



Example: California Housing Data

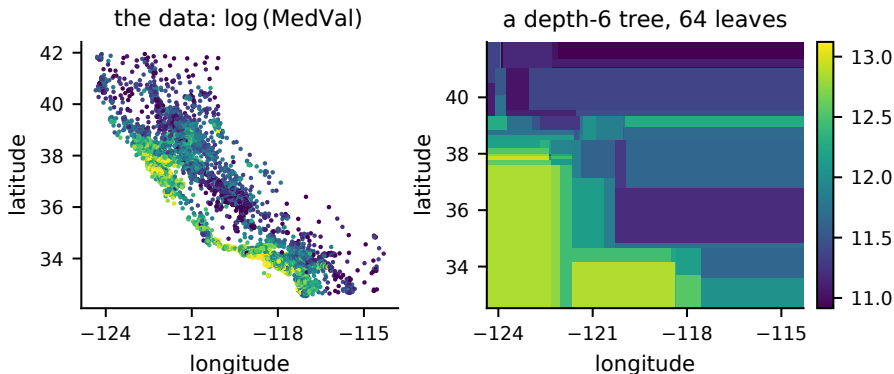
Median home values in 20,640 census tracts, along with

- ▶ Latitude and longitude of tract centers.
- ▶ Population totals, household counts, and median income.
- ▶ Average room/bedroom numbers, home age.

The goal is to predict $\log(\text{MedVal})$ for a census tract.

Difficult regression: covariate effects change with location, and how they change is certainly not linear.

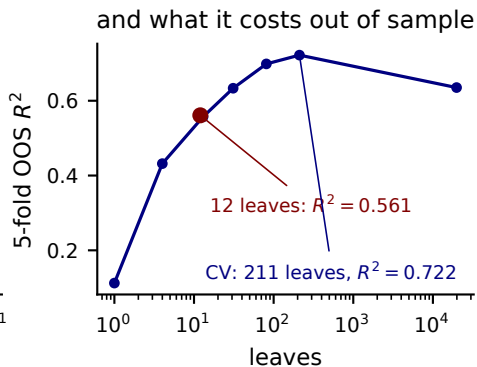
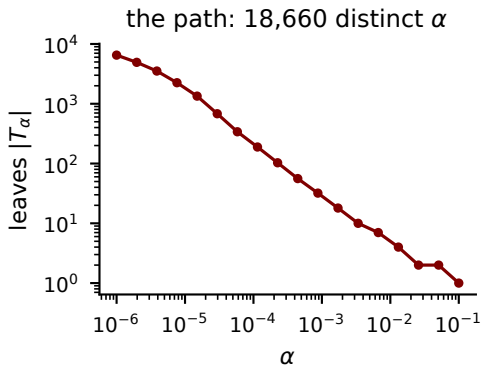
Trees Detect Interaction Automatically



Give a tree **longitude and latitude only** — nothing else, and no instruction that the two interact. Five-fold out-of-sample R^2 : **depth-6 tree 0.5580**, standardized LASSO 0.3164.

A plane in (lat, long) is not a map of California. The tree finds the interaction because recursive splitting *is* interaction.

How Big Should the Tree Be?



Grown out, the tree has 19,826 leaves for 20,640 tracts: it interpolates, in-sample $R^2 = 1.0000$. The cost-complexity path visits 18,660 distinct values of α .

α	0	10^{-4}	3×10^{-4}	10^{-3}	3×10^{-3}	10^{-2}	10^{-1}
leaves	19,826	211	81	31	13	4	1
OOS R^2	0.6352	0.7220	0.6985	0.6334	0.5571	0.4318	0.1124

What the Path Is Telling You

- ▶ Cross-validation picks $\alpha = 10^{-4}$: 211 leaves, out-of-sample $R^2 = 0.7220$. Not the biggest tree, and not a small one either.
- ▶ A twelve-leaf tree — the size you can actually read off a slide — scores 0.5610. That is a legible tree. It is also 0.16 of R^2 worse than the tree cross-validation wants.
- ▶ **The readable tree and the accurate tree are not the same tree.** You cannot have both, and which one you want depends on whether the deliverable is a forecast or an explanation.
- ▶ And note what the left panel says about tuning: leaves fall roughly log-linearly in α , so a coarse geometric grid is enough. You do not need to visit all 18,660 knots.

The Problem Is Not Bias. It Is Variance.

Cross-validated pruning gave us a 211-leaf tree with out-of-sample $R^2 = 0.7220$. That is the best single tree there is, and it is still not good enough.

- ▶ A deep tree can approximate almost any function — grown out, ours interpolated all 20,640 tracts.
- ▶ What it cannot do is give you the **same** function twice. The greedy first split is a choice between close competitors; if a resample flips it, **every split below it changes**.
- ▶ Prediction error decomposes as $(\text{bias})^2 + \text{variance} + \text{noise}$, and for a deep tree the middle term dominates.

Cross-validation picks the depth that minimizes *average* out-of-sample error. It does nothing about the dispersion *at* that depth — and with one history you only get one draw.

Unfortunately, it is tough to avoid overfit with CART:

Deep tree structure is so unstable that optimal depth is not easily chosen via cross validation, and there's no theory to fall back on.

Instead, we can average over a **bootstrapped** sample of trees:

- ▶ repeatedly re-sample the data, **with-replacement**, to get a 'jittered' dataset of n observations.
- ▶ for each resample, **fit a CART tree**.
- ▶ when you want to predict y for some $\mathbf{x}$, take the **average** prediction from this forest of trees.

Real structure that persists across datasets shows up in the average. Noisy useless signals will average out to have no effect.

This is a Random Forest

Random Forests

Algorithm Sketch:

1. Sample B subsets of the data + variables: e.g., observations 1, 5, 20, ... and inputs 2, 10, 17, ...
2. Fit a tree to each subset, to get B fitted trees is $\mathcal{T}_b$. At each split, sample a subset of candidate variables for splitting
3. Average prediction across trees:
 - ▶ for regression average $\mathbb{E}[y|\mathbf{x}] = \frac{1}{B} \sum_{b=1}^B \mathcal{T}_b(\mathbf{x})$.
 - ▶ for classification let $\{\mathcal{T}_b(\mathbf{x})\}_{b=1}^B$ vote on $\hat{y}$.

The observation resample is usually *with-replacement*, so that this is taking the *average of bootstrapped trees* (i.e., 'bagging')

Step 2 Is the Whole Difference

Drop step 2 and you have **bagging**: bootstrap, fit, average. Keep it and you have a **random forest**. The gap between them is worth understanding.

- ▶ **Bagged trees are correlated.** They all see nearly the same data, so they all find nearly the same strong first split. Averaging B estimators cuts variance by about B *only if they are independent*; averaging correlated things buys much less.
- ▶ **Starving each split of columns decorrelates them.** Offer only a random subset of predictors at every split — $\sqrt{p}$ for classification, $p/3$ or p for regression — and the trees are forced to disagree.
- ▶ **The price is a little bias per tree.** Sometimes the best split really was the one you hid. That is a good trade: bias is small and shared, variance is large and cancels.

Understanding Random Forests

Recall how CART is used in practice.

- ▶ Split to lower deviance until leaves hit minimum size.
- ▶ Create a set of candidate trees by pruning back from this.
- ▶ Choose the best among those trees by cross validation.

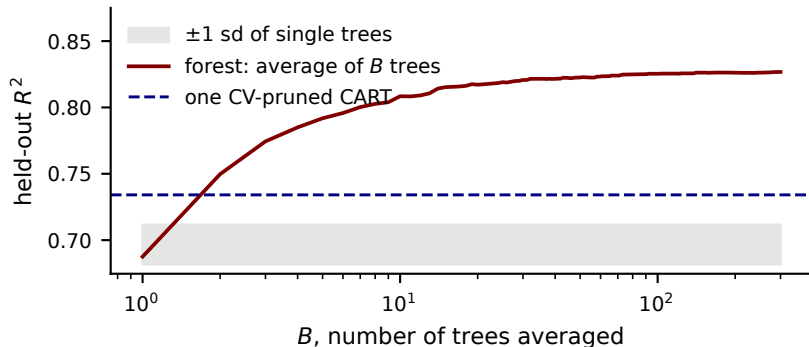
Random Forests avoid the need for CV.

Each tree ' b ' is not overly complicated because you only work with a limited set of variables.

Your predictions are not 'optimized to noise' because they are averages of trees fit to many different subsets.

Each tree is grown deep and *not* pruned — the averaging does the regularizing. In practice you set B as large as your patience allows, set a minimum leaf size, and stop.

What Averaging Actually Buys



California panel, 70/30 split (14,447 fit, 6,193 held out) — a [cross-section](#), so a random split is legitimate.

- ▶ one tree $R^2 = 0.6875 \rightarrow$ ten 0.8084 $\rightarrow$ fifty 0.8225 $\rightarrow$ **three hundred 0.8268**
- ▶ the CV-pruned single CART on the same split: [0.7341](#). Averaging buys +0.0926.
- ▶ the 300 individual trees have held-out R^2 with **sd 0.0148** about their mean — and each is a legitimate CART fit.

Flat after about fifty trees: B is a budget, not a tuning parameter.

Model Averaging

This technique of 'Model Averaging' is central to many advanced nonparametric learning algorithms.

ensemble learning, mixture of experts, Bayesian averages, ...

It works best with flexible but simple models

Recall lasso as a stabilized version of stepwise regression (if you jitter the data your estimates stay pretty constant).

Model averaging is a way to take arbitrary *unstable* methods, and make them stable. **This makes training easier.**

Probability of rain on a new day is the average $P(\text{rain})$ across some trees that split on forecast, others on sky. We don't get tied to one way of deciding about umbrellas.

Random Forests in python

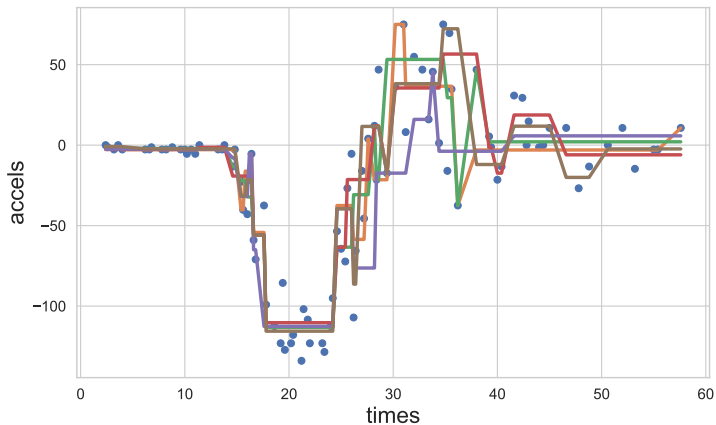
Python has the `RandomForestRegressor` and `RandomForestClassifier` in `sklearn`, which work essentially the same as `tree`.

```
rf = RandomForestRegressor(n_estimators=300).fit(X, y)
```

Unfortunately, you lose the interpretability of a single tree. You have traded one readable rule set for three hundred of them.

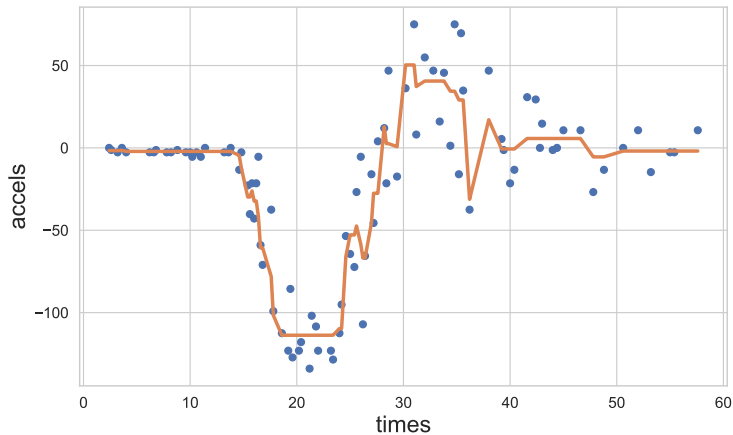
One warning about `feature_importances_`, because the name invites the wrong reading. It is the **in-sample** mean decrease in impurity, summed over splits and normalized. It is not an out-of-sample statistic. If you want importance measured out of sample, permute — we do that at the end of the lecture, and the two rankings disagree.

Random Trees for the Motorcycle Data



If you fit to random subsets of the data, you get a slightly different tree each time.

Model Averaging with Random Forests



Averaging many trees yields a single response surface.

Ensemble Learning with Boosted Trees

“**Boosting**”: Ensemble method for aggregating many individual “**weak learners**” (models that don’t forecast well) into a single “**strong learner**”. Cf. [Schapire \(1990, ML\)](#)

Algorithm Sketch:

- ▶ Fix tree-depth L (shallow!, e.g. $L = 1$)
 1. Fit 1st simple tree
 - ▶ Forces trees to be weak learners with large bias
 2. Fit 2nd shallow tree to **residuals from the first tree**
 - ▶ Forecasts from the two trees are summed to form the ensemble prediction
 - ▶ Shrink the forecast component from the 2nd tree by a factor $\nu \in (0, 1)$
 - ▶ Helps prevent the model from overfitting the residuals
- ▶ Iterate until there are a total of B trees in the ensemble

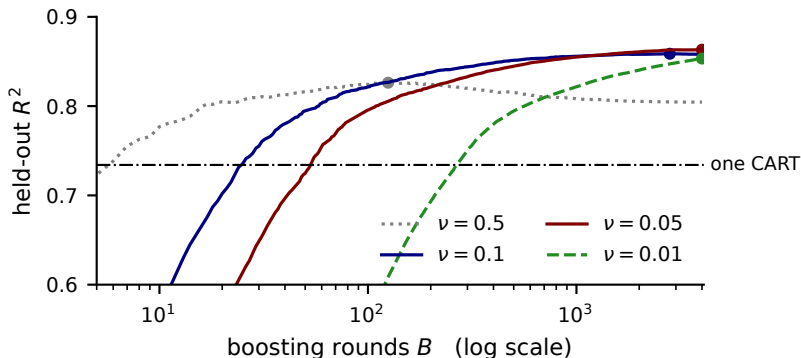
L , ν , and B are tuning parameters selected via validation.

Bagging and Boosting Are Opposites

Bagging / random forest	Boosting
Trees are fit in parallel , each to a resample of the same data.	Trees are fit in sequence , each to what the previous ones got wrong.
Trees are deep : low bias, high variance. Averaging kills the variance.	Trees are shallow : high bias, low variance. Adding kills the bias.
Trees should be as independent as possible — hence column subsampling.	Trees are deliberately dependent — tree b exists only because of trees $1..b-1$.
More trees never hurt. B is a budget.	More trees eventually hurt. B is a tuning parameter.

Same ingredient, opposite recipe. That last row is why boosting needs more care than a forest — and why it wins when you give it that care.

Shrinkage, Rounds, and When to Stop



Held-out R^2 against rounds; California panel, depth-4 trees, subsample 0.8:

- ▶ $\nu = 0.5$: peaks at 125 rounds, 0.8262, then **decays** to 0.8045 by 4,000. Big steps overfit the residual.
- ▶ $\nu = 0.1$: peaks at 2,803, 0.8586. $\nu = 0.05$: 0.8632, still creeping up at 4,000. $\nu = 0.01$: 0.8533, not converged.

Halve the learning rate and you need roughly twice the rounds. Pick ν small enough to be safe, then let the held-out curve tell you B .

Back to California Housing

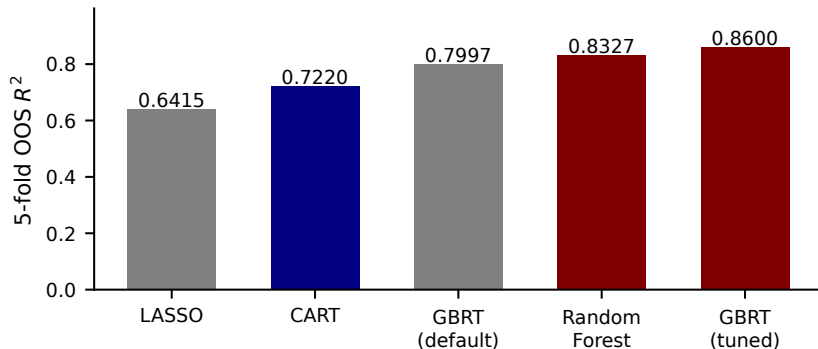
Same 20,640 census tracts as Part I, same nine predictors, same target `log(MedVal)`.

Five models, all scored on the **same** 5-fold split, against the same training-fold mean:

- ▶ standardized LASSO — the linear benchmark
- ▶ cross-validated CART — one tree, 211 leaves
- ▶ `GradientBoostingRegressor(100)` — boosting straight out of the box
- ▶ `RandomForestRegressor(300)` — the forest
- ▶ boosting with ν and B chosen on held-out data

The comparison is only meaningful because the split is shared. Two numbers computed on two different splits are two different questions.

CA Housing: Out-of-Sample Prediction



- ▶ **One tree beats the linear model:** 0.7220 against 0.6415. The gap is nonlinearity plus interaction — the geographic map of Part I, in a number.
- ▶ **Averaging beats one tree:** 0.8327 against 0.7220, for almost no tuning.
- ▶ **Boosting out of the box *loses to the forest*,** 0.7997. One hundred rounds at the default rate is not enough.
- ▶ **Boosting tuned wins,** 0.8600 — bought with 3,998 rounds and a held-out search, where the forest cost one line.

The Honest Version of “Boosting Wins”

Boosted trees are the standard workhorse on tabular data, and that reputation is earned. But state it precisely:

- ▶ A random forest is nearly tuning-free. Set B large, set a minimum leaf size, walk away. On this panel that gets you 0.8327.
- ▶ Boosting is not. With ν , B , depth and subsampling all live, the out-of-the-box call sits 0.03 *below* the forest. The win of +0.027 is real, and it is bought with a validation budget.
- ▶ And a tuning budget is a leakage risk. Every round of the search looks at held-out data. Tune on a validation split, report on a test split you touched once.

If you have a day, boost. If you have an hour, or a deadline, or a committee, the forest is the better answer and it is not close.

Variable Importance

The mean decrease in impurity (MDI) importance of a feature is computed as follows:

1. For each tree in the forest:
 - ▶ The decrease in variance for a feature used in a split is calculated every time that feature is used to split a node.
 - ▶ The decrease is a measure of how much that split improves the homogeneity of the node, specifically, how much the impurity of the target variable is reduced as a result of the split.
 - ▶ This decrease in impurity is averaged across all the splits in the tree that use the feature.
2. For the entire forest:
 - ▶ The average decrease in variance for each feature is then averaged across all the trees in the forest to give a final importance score for each feature.
3. Normalization:
 - ▶ The feature importance scores are then normalized so that they sum to 1.

Note what this is not: every quantity above is computed on the rows the tree was fit to. MDI is an in-sample statistic.

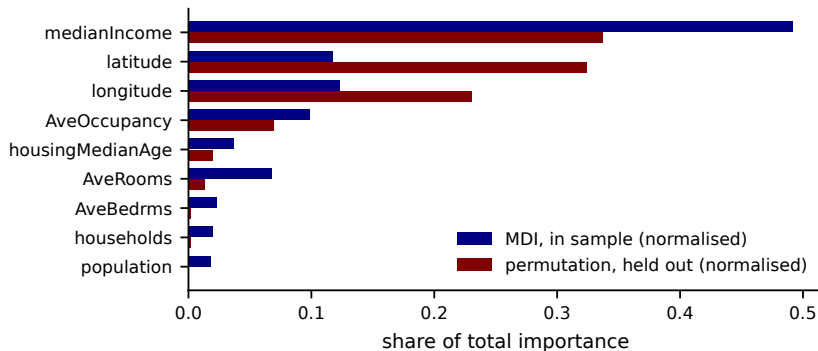
Permutation Variable Importance

Permutation-based feature importance is model-agnostic and is often more reliable, especially in the presence of correlated features. This method involves the following steps:

1. Train the model on the original dataset and compute the performance metric of interest on held-out data.
2. For each feature:
 - ▶ Shuffle the values of the feature across all samples, thereby breaking the relationship between the feature and the target.
 - ▶ Recompute the performance metric of the model using this modified dataset.
 - ▶ The importance of the feature is how much the metric *decreases*. A larger decrease indicates a more important feature.
3. Repeat the shuffle several times and average, since a single permutation is itself a random draw.

Permutation importance is more computationally intensive than MDI because it requires re-evaluating the model for each feature — and it is worth it, because it answers the question you actually asked.

The Two Rankings Disagree



Same forest, same data; MDI in sample, permutation on the held-out third:

- **Latitude:** MDI third at 0.1177, permutation **second** at 0.6101. MDI splits the credit for location across two correlated columns and undercounts both.
- **Population and households:** real MDI mass (0.0185, 0.0201), worth essentially nothing out of sample (0.0015, 0.0035). MDI rewards a column for merely offering many places to split.

Partial Dependence, Defined

Take one variable x_j and a value v . Set $x_j = v$ for every row, leave every other column as observed, predict, and average:

$$\bar{f}_j(v) = \frac{1}{n} \sum_{i=1}^n \hat{f}(v, \mathbf{x}_{i,-j}).$$

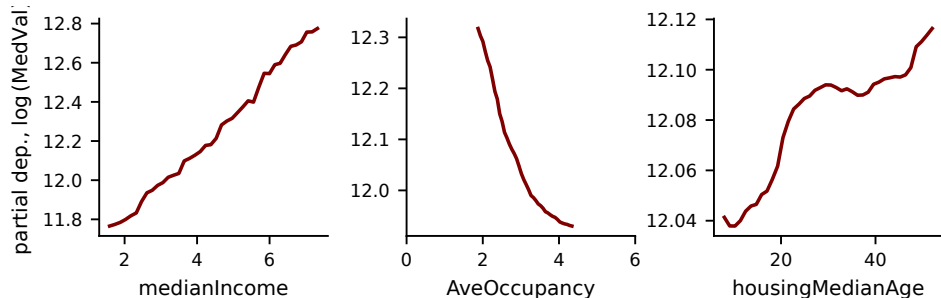
Repeat over a grid of v — forty points from the 5th to the 95th percentile of x_j — and plot $\bar{f}_j(v)$ against v . That is the whole method:

```
def partial_dependence(model, X, feature, grid):
    out = []
    for v in grid:
        Xv = X.copy()
        Xv[feature] = v          # every row gets x_j = v; the rest as observed
        out.append(model.predict(Xv).mean())
    return np.array(out)
```

It averages over the other columns *as observed*, so it reports what the model says, not a causal effect.

`sklearn`'s `partial_dependence` defaults to a faster tree-walk approximation for forests; `method='brute'` is this definition exactly.

Importance Says Which. Partial Dependence Says Which Way.



Each panel is $\bar{f}_j(v)$ from the previous slide, computed on the fitting sample with the seven-line function.

- ▶ **medianIncome** spans 1.01 in log MedVal — a factor of 2.7 in levels — and rises throughout. A sentence a committee can act on.
- ▶ **AveOccupancy** spans 0.39, steeply decreasing at the low end then nearly flat. One linear coefficient would average those two regimes into a misleading number.
- ▶ **housingMedianAge** spans 0.08: present, economically small.

Importance is a magnitude with no sign. Partial dependence has the sign.

Roundup on Tree-based Learning

We have seen three techniques for building tree models.

- ▶ **CART:** recursive partitions, pruned back by CV.
Readable. Unstable. On the California panel, $R^2 = 0.7220$.
- ▶ **Random Forest:** average many deep CART trees, decorrelated by column subsampling.
Nearly tuning-free, and $R^2 = 0.8327$. The default answer.
- ▶ **Boosted Trees:** repeatedly fit shallow trees to residuals, shrinking each by ν .
Best in class at 0.8600 — *after* ν and B are chosen honestly. Out of the box it loses to the forest.

Trees are poor in high dimension, but fitting them to low-dimensional factors (principal components, Lecture 1) is a good option.

And whichever you pick, you have bought accuracy with interpretability. Permutation importance and partial dependence are how you buy some of it back.

Roundup on Nonlinear Regression and Classification

Many other *nonparametric learning* algorithms

- ▶ Neural Networks (and deep learning):
many recursive logistic regressions.
- ▶ Support Vector Machines:
Project to HD, then classify.
- ▶ Gaussian Processes, splines, wavelets, etc:
Use sums of curvy functions in regression.

Some of these are great, but all take a ton of tuning.

On tabular financial data of the size you will actually meet, nothing out of the box beats a tree ensemble. When the linear model does win, it is usually because the signal really is weak and linear — which, in asset pricing, is more often than you would like.